Medplum Healthcare Platform on Azure Cloud

Final Project Report

Project Title: Cloud Infrastructure Deployment for Medplum Healthcare Platform
Institution: EPITA
Group Members: Muhammad Naveed, Naveed Riaz, Muhabbat Ali, Muntazir Abbas

Cloud Provider: Microsoft Azure

Table of Contents

1. Executive Summary
2. Project Overview
3. Architecture Design
4. Azure Resources Implementation
5. Deployment Process
6. Security Implementation
7. Cost Analysis
8. Challenges and Solutions
9. Future Improvements
10. Conclusion

---

1. Executive Summary

This project successfully deployed a complete healthcare platform (Medplum) on Microsoft Azure using modern Infrastructure as Code (IaC) practices. The deployment utilizes Azure Kubernetes Service (AKS) as the container orchestration platform, integrated with various Azure services to create a scalable, secure, and cost-effective solution.

Key Achievements:

- Deployed 8 containerized services on Azure Kubernetes Service
- Configured automated infrastructure provisioning using Terraform
- Implemented secure networking with VNet and Network Security Groups
- Established DNS management for public access via custom domain
- Set up monitoring and logging infrastructure
- Achieved production-ready deployment with persistent storage

---

2. Project Overview

2.1 Project Goals

The primary objective was to deploy a healthcare platform on Azure cloud infrastructure with the following requirements:

1. **Scalability**: Ability to handle varying loads through auto-scaling
2. **High Availability**: Multiple replicas and automatic failover
3. **Security**: Network isolation, access controls, and encrypted communications
4. **Cost Efficiency**: Optimized resource usage to minimize operational costs
5. **Maintainability**: Infrastructure as Code for reproducible deployments
6. **Monitoring**: Comprehensive observability and alerting
7. **Data Backups (Future Plan)**: Take a regular snapshots of data and store in Azure Storage.

2.2 Technology Stack

Infrastructure Tools:

- **Terraform**: Infrastructure as Code for Azure resource provisioning
- **Bash Scripting**: Configuration management and service deployment
- **Helm**: Kubernetes package manager for complex applications

Azure Services:

- Azure Kubernetes Service (AKS)
- Azure Container Registry (ACR)
- Virtual Machine Scale Set (VMSS)
- Azure Virtual Network
- Azure Blob Storage
- Azure DNS
- Azure Load Balancer
- Network Security Group
- Resource Group
- Access Control IAM
- Azure Monitor & Log Analytics

Application Components:

- Frontend (React-based UI)
- Backend API (Node.js/REST API)
- GraphQL API
- PostgreSQL Database
- Portainer (Container Management)
- Prometheus (Metrics Collection)

- Kafka (Message Broker)
- Vault (Secrets Management)

---

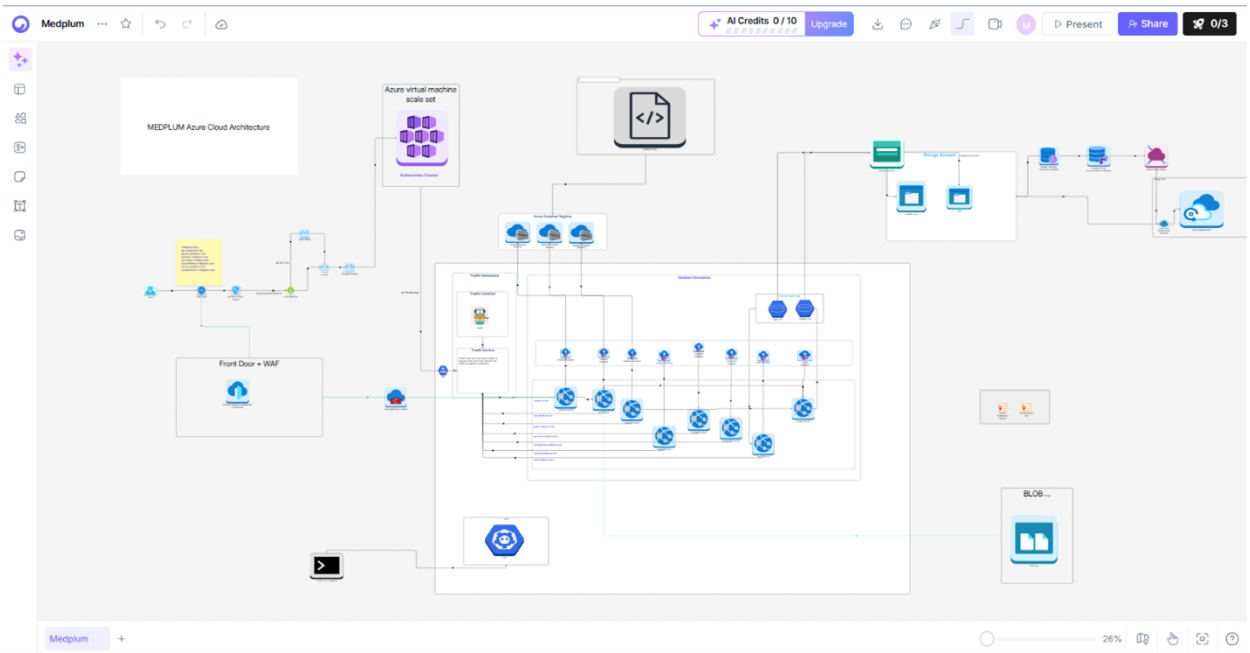
3. Architecture Design

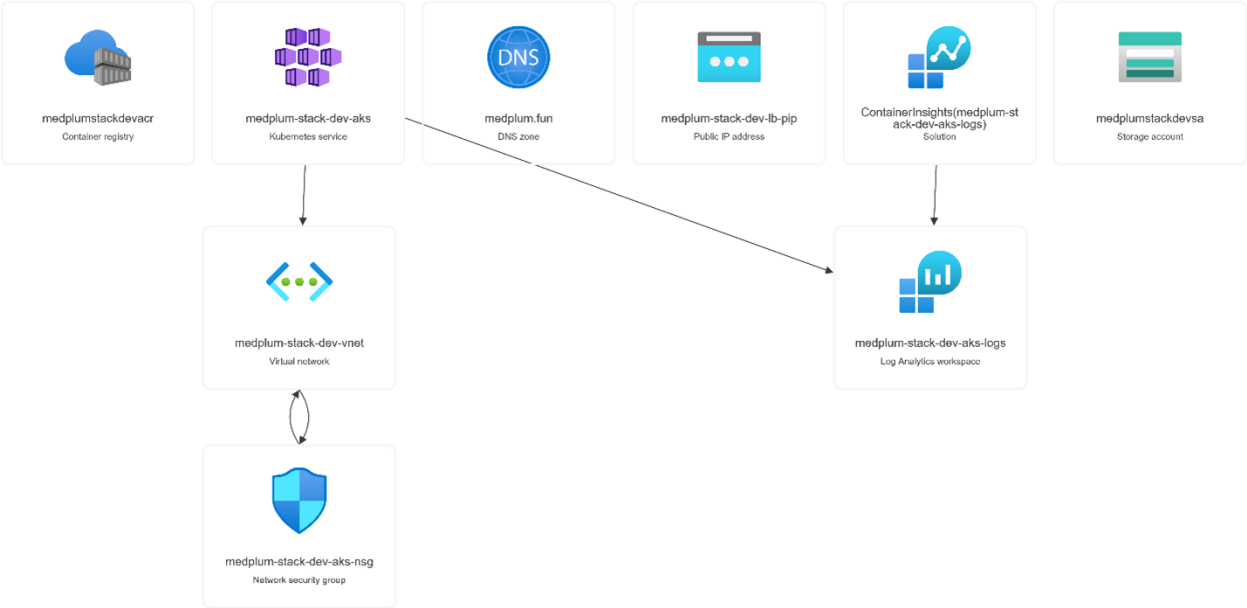
3.1 High-Level Architecture

```
                    Internet
                       ↓
                [Azure DNS Zone]
              (medplum.fun domain)
                       ↓
              [Azure Virtual Network]
            (public & private subnets)
                       ↓
               [Azure Load Balancer]
              (Public IP: 48.194.13.54)
                       ↓
┌──────────────────────────────────────────────┐
│   Azure Virtual Network (VNet)               │
│        (10.0.0.0/16)                         │
│                                              │
│   ┌──────────────────────────────────┐       │
│   │  AKS Subnet                      │       │
│   │  (10.0.1.0/24)                   │       │
│   │                                  │       │
│   │  [AKS Cluster]                   │       │
│   │   ┌──────────────────────┐       │       │
│   │   │ Medplum NS           │       │       │
│   │   │ - Frontend           │       │       │
│   │   │ - Backend            │       │       │
│   │   │ - GraphQL            │       │       │
│   │   │ - PostgreSQL         │       │       │
│   │   │ - Portainer          │       │       │
│   │   │ - Vault              │       │       │
│   │   └──────────────────────┘       │       │
│   │                                  │       │
│   │   ┌──────────────────────┐       │       │
│   │   │ Traefik NS           │       │       │
│   │   │ - Traefik            │       │       │
│   │   └──────────────────────┘       │       │
│   └──────────────────────────────────┘       │
│                                              │
│   ┌──────────────────────────────────┐       │
│   │  Storage Subnet                  │       │
│   │  (10.0.2.0/24)                   │       │
│   └──────────────────────────────────┘       │
└──────────────────────────────────────────────┘
                       ↓
```

[Azure Blob Storage]
[Azure Managed Disks]

Architecture Design (Diagram)

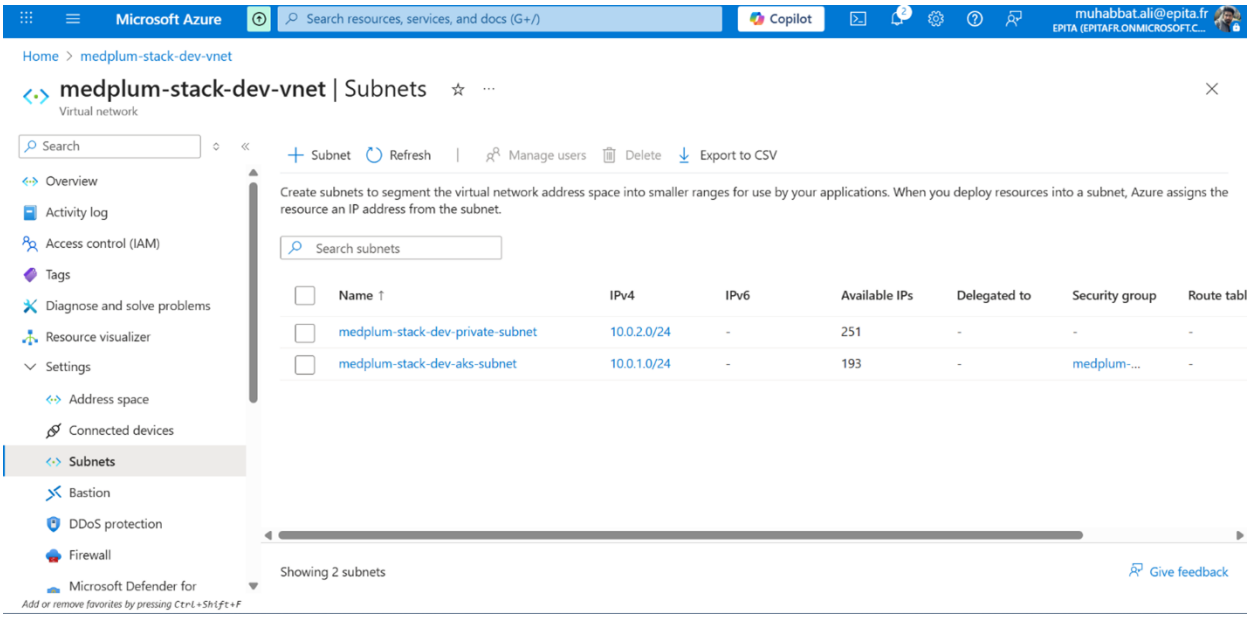




.2 Network Architecture

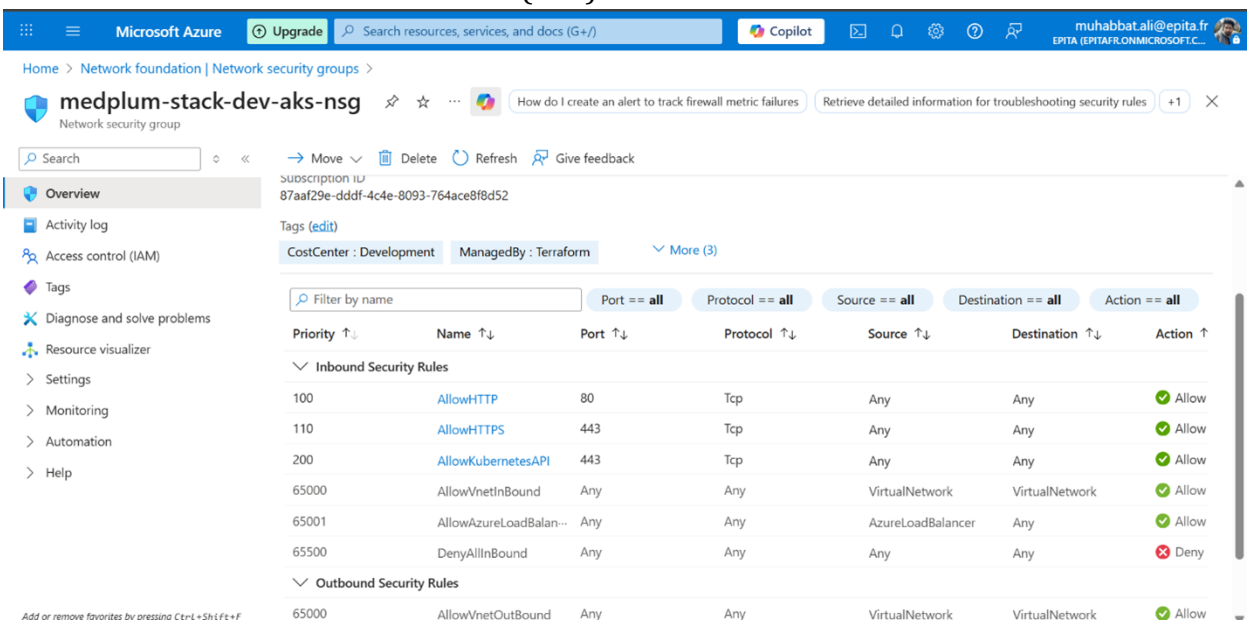
Virtual Network (VNet)

- **Address Space**: 10.0.0.0/16
- **Subnets**:
  - AKS Subnet: 10.0.1.0/24 (for Kubernetes nodes and pods)
  - Public Subnet: 10.0.2.0/24



Network Security

- Network Security Groups (NSG) with allow rules for:
  - Allow inbound HTTP/HTTPS (80, 443)
  - Allow inbound K8s-API (443)



3.3 Service Endpoints

All services are accessible via subdomains.

Service	Subdomain	Port	Purpose
Frontend	app.medplum.fun	3000	User Interface
Backend	api.medplum.fun	8080	REST API
GraphQL	graph.medplum.fun	8000	GraphQL API
Portainer	portainer.medplum.fun	9000	Container Management
Prometheus	prometheus.medplum.fun	9090	Monitoring
Jenkins	jenkins.medplum.fun	7000	CICD piples
Vault	vault.medplum.fun	8200	Secrets Management
PostgreSQL	Internal only	5432	Database

---

4. Azure Resources Implementation

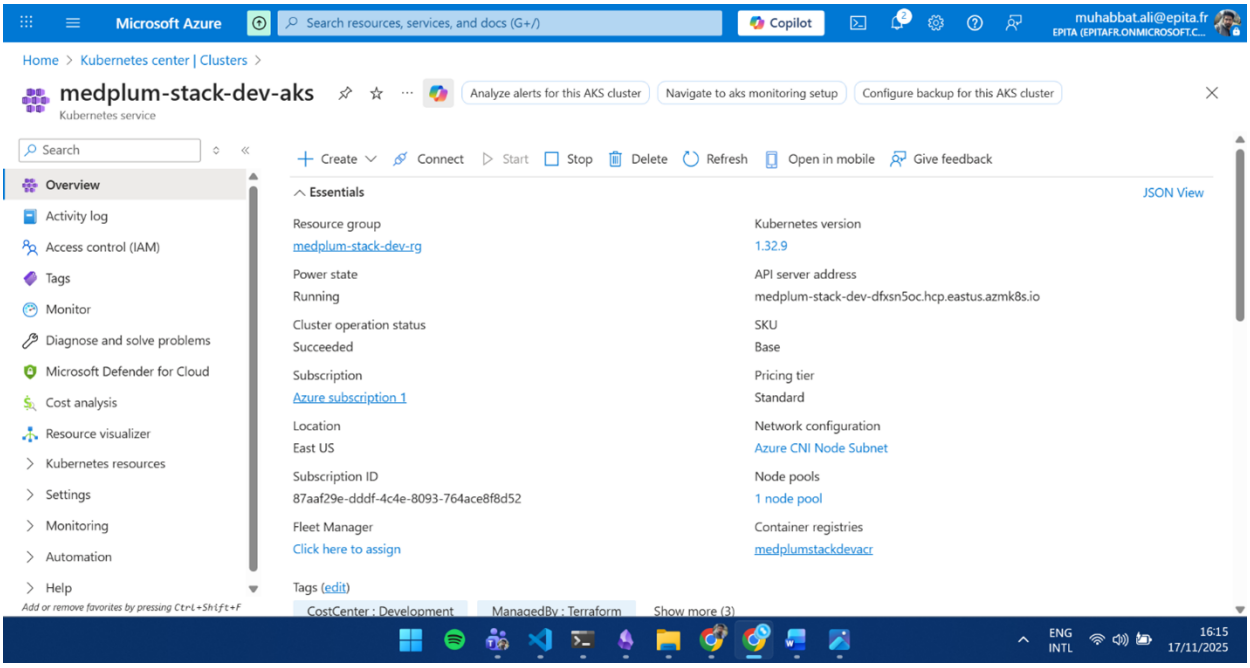
4.1 Resource Group

Resource Group Name: `medplum-stack-dev-rg`
Location: US East
Purpose: Logical container for all project resources

All Azure resources are organized within this single resource group for:

- Simplified management
- Consolidated billing
- Easy resource lifecycle management
- Role-based access control (RBAC) boundaries

4.2 Azure Kubernetes Service (AKS)



Cluster Name: `medplum-stack-dev-aks`
Kubernetes Version: 1.32.9
Node Configuration:

- VM Size: Standard_B2s (2 vCPUs, 4 GB RAM)
- Initial Node Count: 2
- Auto-scaling: Enabled (Min: 1, Max: 2)
- OS Disk Size: 20 GB per node

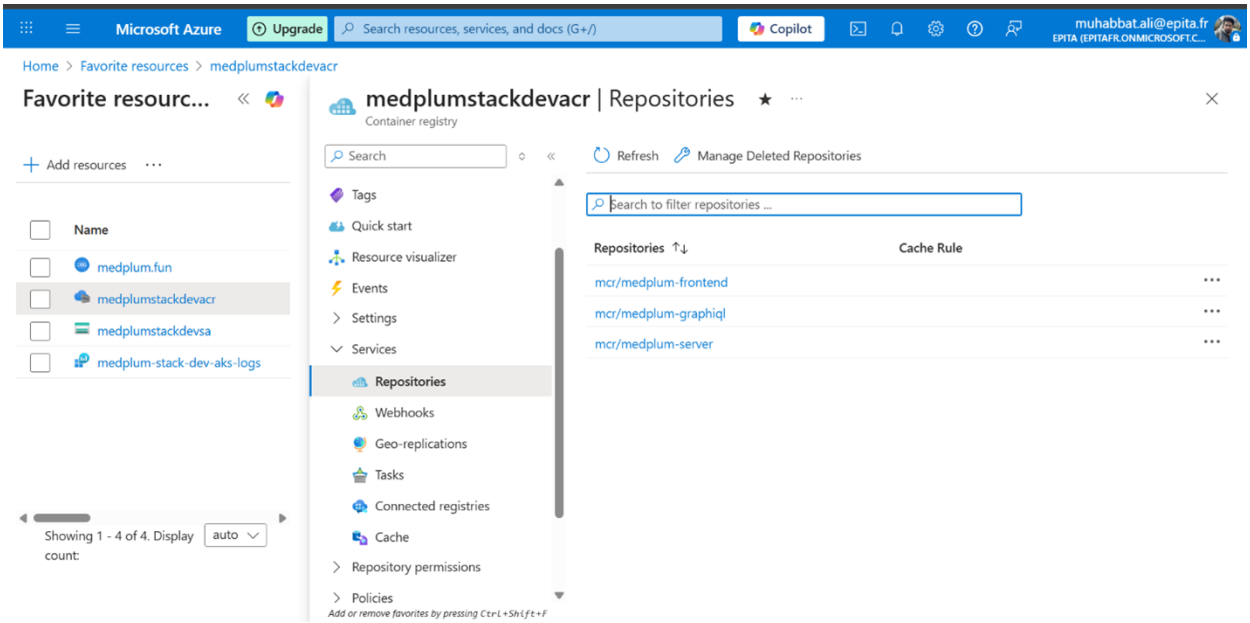
Features Enabled:

- **Auto-scaling**: Dynamically adjusts node count based on workload
- **Azure Monitor Integration**: Container insights for metrics and logs
- **Managed Identity**: System-assigned identity for Azure resource access
- **Azure CNI Networking**: Advanced networking for VNet integration

Why AKS?

- Fully managed Kubernetes service (no master node management)
- Native Azure integration
- Built-in security and compliance features

4.3 Azure Container Registry (ACR)



Registry Name: `medplumstackdevacr`
SKU: Basic
Location: US East

Repositories:

1. **medplum-frontend**: Frontend application images
2. **medplum-backend**: Backend API images
3. **medplum-graphql**: GraphQL API images

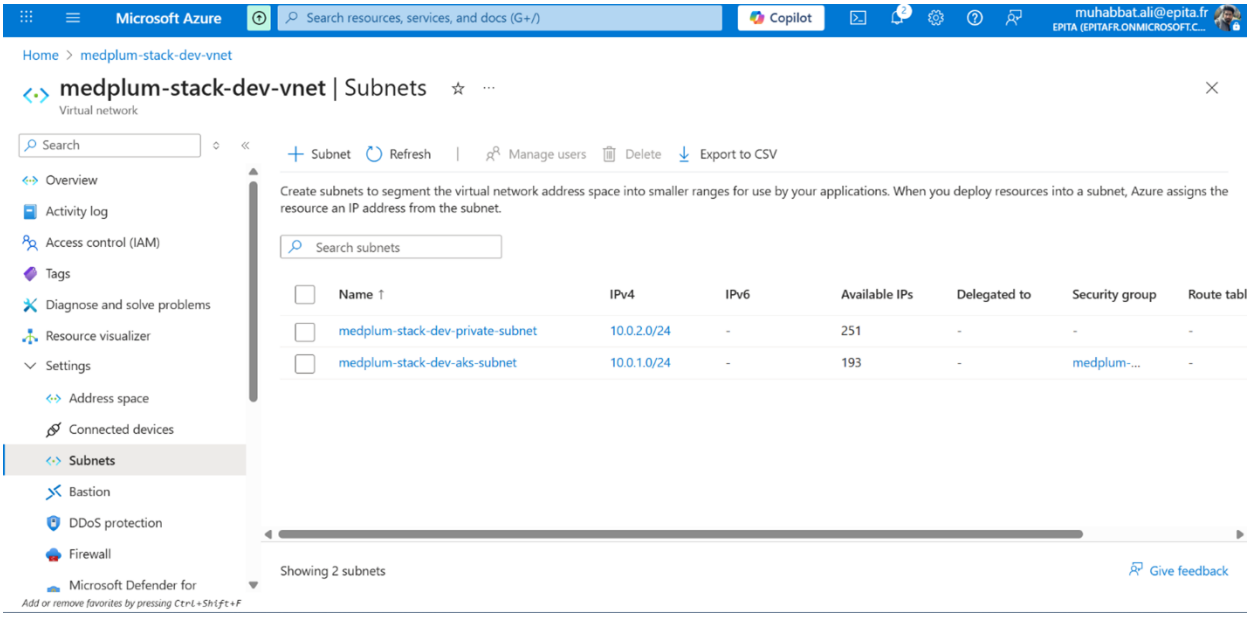
Features:

- **Admin Access Enabled**: For Azure DevOps pipeline authentication
- **AKS Integration**: Direct pull access via managed identity
- **Image Storage**: Automatic versioning with tags (latest, build-id)

Image Management:

- Images are pushed from main branch via bash scripts (Jenkins will be used in future)
- Tagged with build numbers for version tracking
- 'latest' tag always points to most recent build

4.4 Azure Virtual Network (VNet)



VNet Name: `medplum-stack-dev-vnet`
Address Space: 10.0.0.0/16
Location: France Central

![VNet Screenshot - To be inserted]

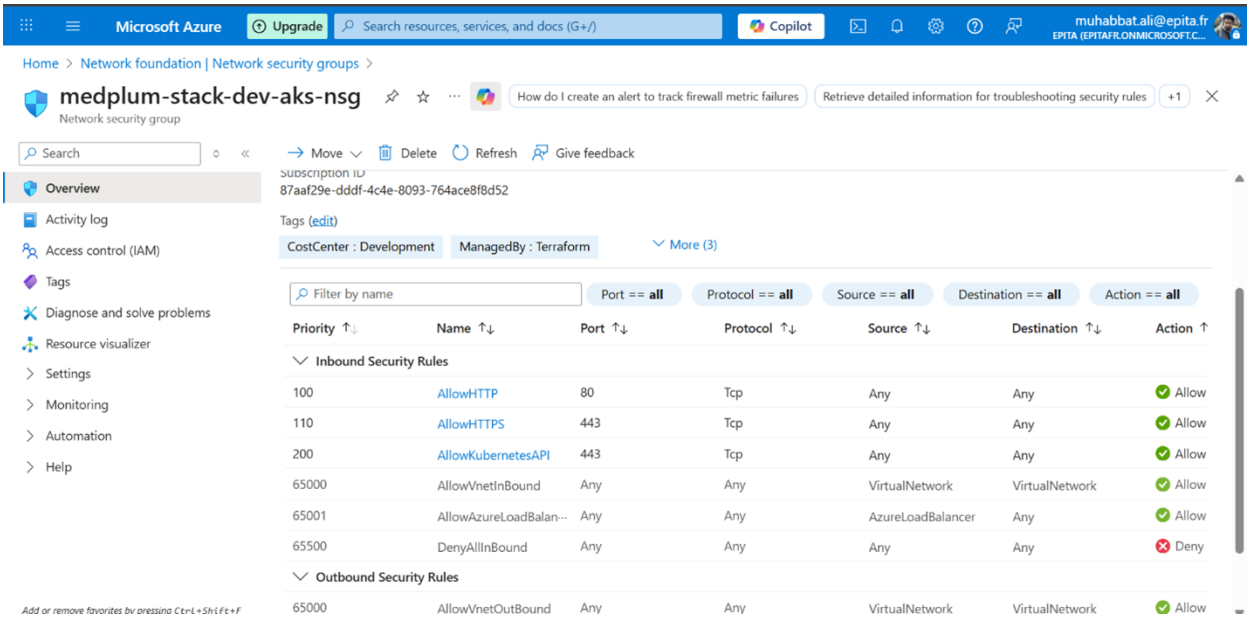
Subnet Configuration:

Subnet Name	Address Range	Purpose	Connected Resources
aks-subnet	10.0.1.0/24	AKS nodes	AKS Cluster
storage-subnet	10.0.2.0/24	Storage services	Blob Storage endpoint

Network Features:

- **Service Endpoints**: Enabled for Azure Storage (secure access)
- **DNS**: Custom DNS settings pointing to Azure DNS
- **Network Policies**: Azure CNI network policies enabled

4.5 Network Security Group (NSG)



NSG Name: `medplum-stack-dev-aks-nsg`
Associated Subnet: aks-subnet

![NSG Screenshot - To be inserted]

Security Rules (Inbound):

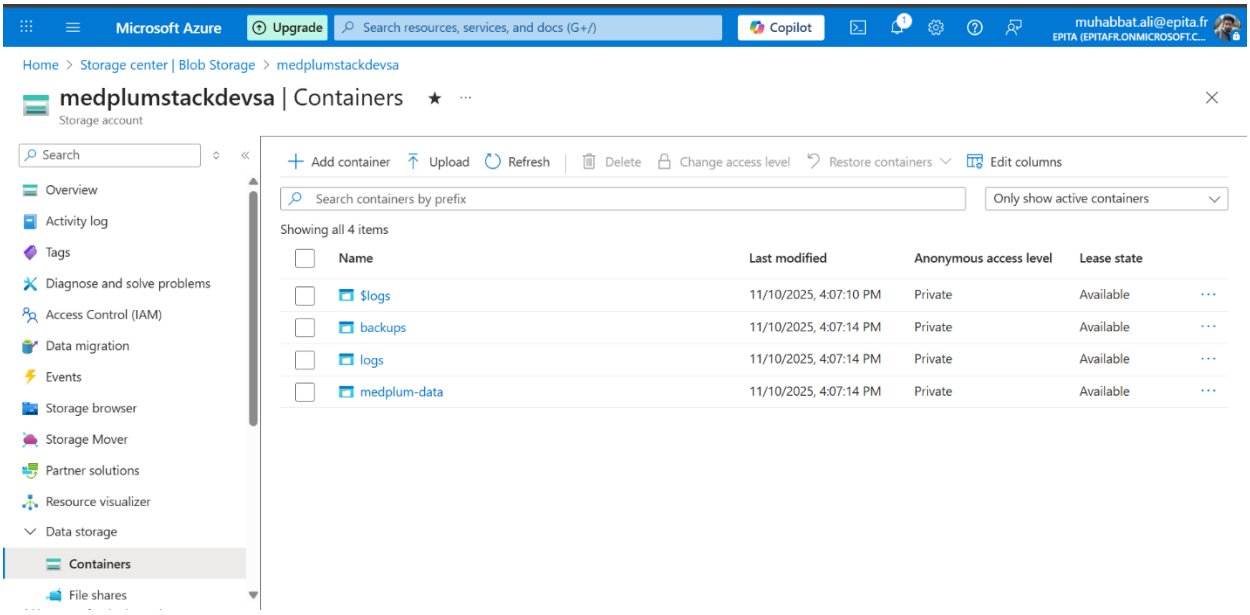
Priority	Name	Port	Protocol	Source	Description
100	Allow-HTTP	80	TCP	Any	Public HTTP access
110	Allow-HTTPS	443	TCP	Any	Public HTTPS access
180	Allow-K8s-API	443	TCP	Any	Kubernetes API

Security Considerations:

- Default deny for all other traffic
- Outbound rules allow necessary Azure service communication
- Can be further restricted by source IP ranges in production

---

4.6 Azure Storage



Storage Account Name: `medplumstackdevsa`
Account Type: Standard LRS (Locally Redundant Storage)
Location: East US

Containers:

1. **medplum-data**: Application data storage
2. **backups**: Database and configuration backups

Configuration:

- **Minimum TLS Version**: 1.2 (secure connections only)
- **Public Access**: Disabled (private access only)
- **Network Rules**: Access restricted to VNet subnets
- **Versioning**: Disabled (cost optimization)

Access Method:

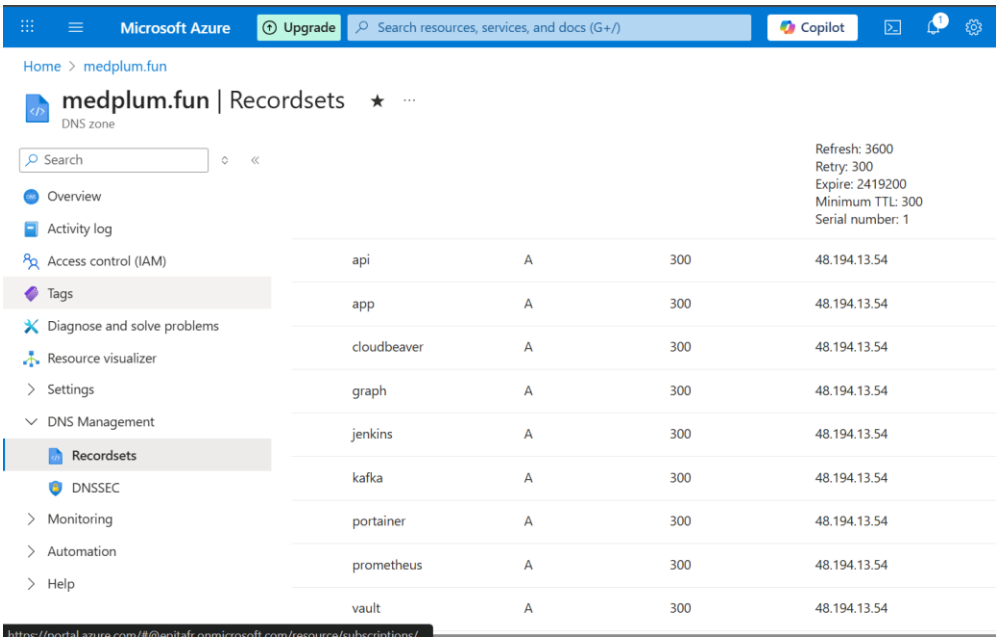
- Kubernetes pods access via service endpoint
- Connection strings stored in Kubernetes secrets
- Firewall rules limit access to AKS subnet only

4.7 Azure DNS Zone

Zone Name: `medplum.fun`
Resource Group: medplum-stack-dev-rg

DNS Records (A Records):

Name	Type	TTL	Value	Purpose
api	A	300	48.194.13.54	Backend API
app	A	300	48.194.13.54	Frontend
graph	A	300	48.194.13.54	GraphQL
portainer	A	300	48.194.13.54	Management UI
prometheus	A	300	48.194.13.54	Monitoring
vault	A	300	48.194.13.54	Secrets



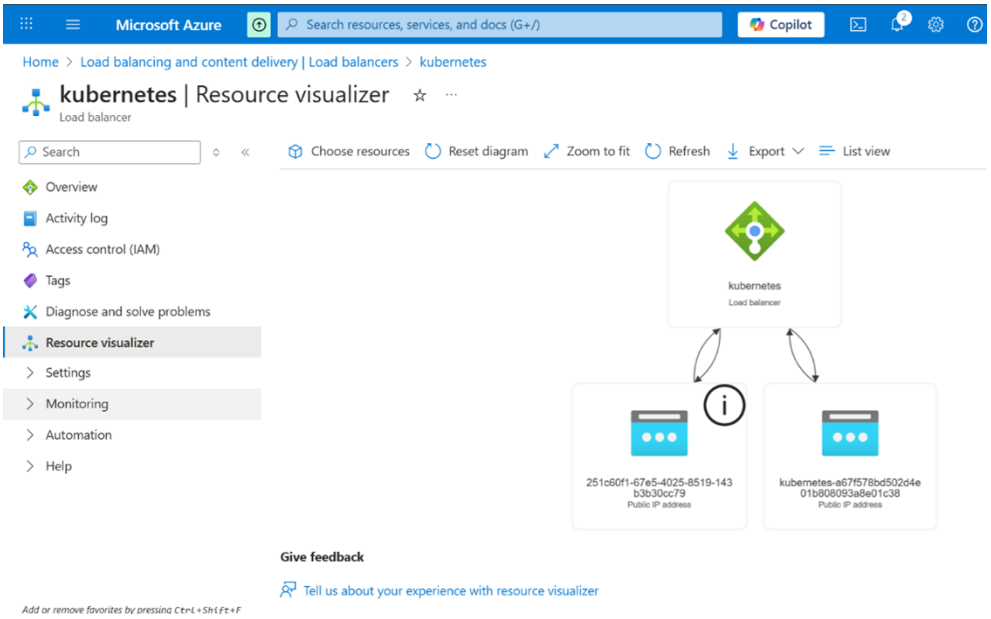
DNS Configuration:

- Domain registered at Hostinger
- Nameservers delegated to Azure DNS
- All records point to Azure Load Balancer public IP
- TTL set to 300 seconds for faster updates

Nameserver Delegation: The domain medplum.fun nameservers at Hostinger were updated to point to Azure DNS nameservers provided by Azure.

4.8 Azure Load Balancer

Load Balancer Name: Auto-created by AKS
Type: Standard SKU
Public IP: `medplum-stack-dev-lb-pip`



Configuration:

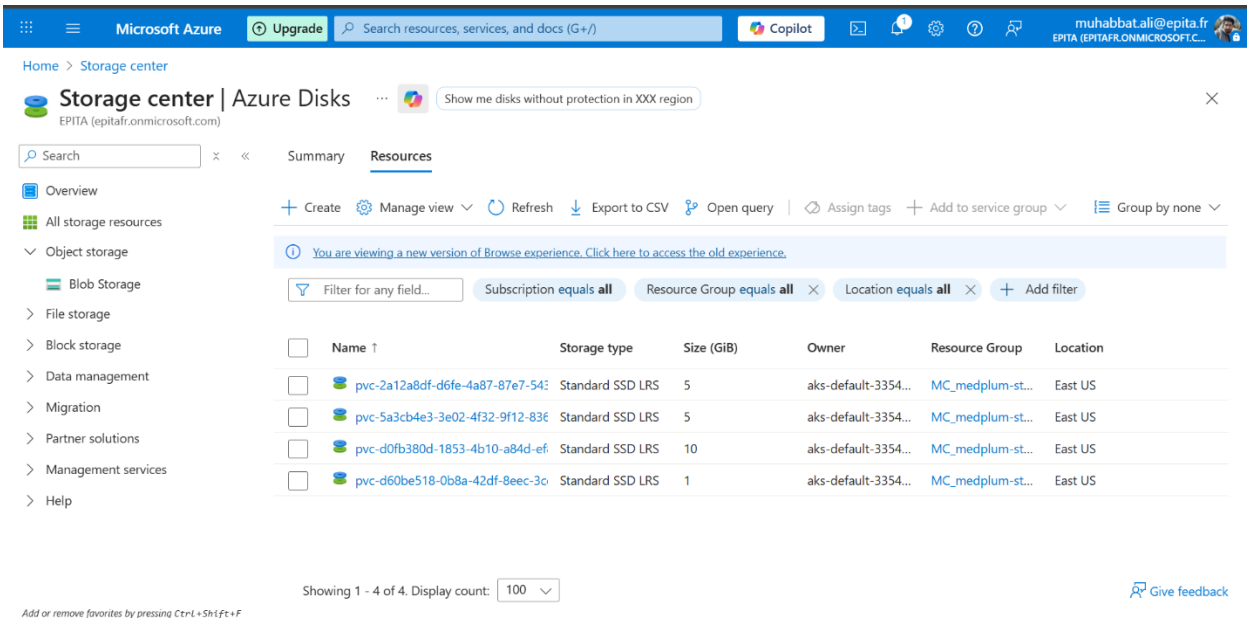
- **Allocation Method**: Static (fixed public IP)
- **SKU**: Standard (required for AKS)
- **Backend Pool**: AKS nodes
- **Load Balancing Rules**: Automatic (managed by Kubernetes)

How It Works:

1. Traefik service is exposed with type "LoadBalancer" on IP 48.194.13.54
2. Each service is medplum namespace is mapped by Traefik Router for external discovery
3. Azure Load Balancer distributes traffic to healthy backend pods
4. Health probes ensure traffic only goes to running containers
5. Session affinity can be configured per service if needed

4.9 Azure Managed Disks

Disk Type: Premium SSD (for AKS nodes) and Standard SSD (for PVCs)



Usage:

1. **AKS Node OS Disks**: 5 GB per node (2 disks for 2 nodes)
2. **PostgreSQL PVC**: 10 GB persistent volume for database
3. **Prometheus PVC**: 1 GB for metrics storage

Storage Class: `azure-disk-standard`

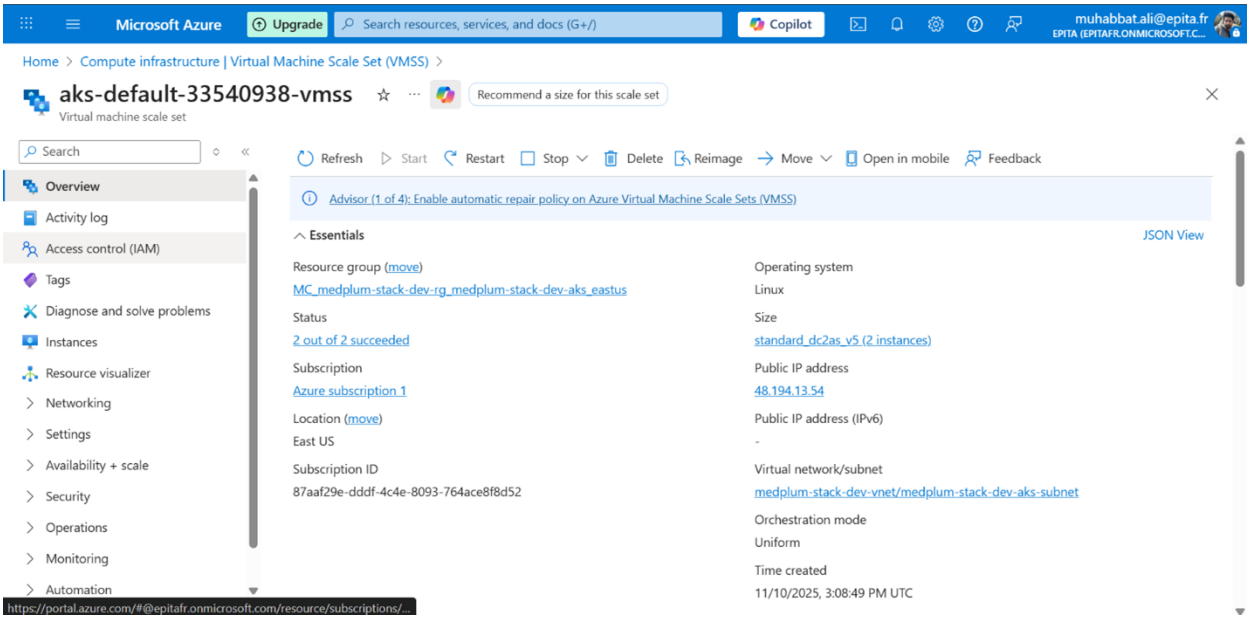
- **Provisioner**: disk.csi.azure.com (Azure Disk CSI driver)
- **Reclaim Policy**: Retain (data preserved on pod deletion)
- **Volume Binding Mode**: WaitForFirstConsumer (disk created in same zone as pod)

Benefits:

- Automatic provisioning via Kubernetes
- Data persists across pod restarts
- Snapshot capability for backups
- Automatic encryption at rest

4.10 VMSS (Virtual Machine Scale Set)

This is the virtual machine that runs the Kubernetes cluster. It was automatically created when terraform created the cluster.



4.11 Log Analytics Workspace

Workspace Name: `medplum-stack-dev-logs`
Retention: 30 days
SKU: PerGB2018 (pay-as-you-go)

Data Sources:

- AKS Container Logs
- AKS Metrics
- Node performance data
- Kubernetes events
- Application logs

Queries Available:

- Container CPU and memory usage
- Pod restart counts
- Error log aggregation
- Network traffic analysis
- Performance troubleshooting

Integration: Connected to Azure Monitor for:

- Custom dashboards
- Alert rules

4.12 Azure Monitor

Monitoring Coverage:

- AKS cluster health metrics
- Node performance (CPU, memory, disk, network)
- Pod metrics and status
- Container insights
- Custom application metrics via Prometheus

![Azure Monitor Screenshot]

Key Metrics Tracked:

- Node CPU percentage
- Node memory usage
- Pod count per namespace
- Container restart counts

Alerting (Future Implementation):

- High CPU usage (>80% for 5 minutes)
- High memory usage (>85%)
- Pod crash loops
- Node not ready status
- Disk space warnings

---

5. Deployment Process

5.1 Infrastructure Deployment (Terraform)

Step 1: Prerequisites Installation

```
# Install required tools
- Terraform >= 1.5.0
- Azure CLI
- kubectl
- Helm
```

Step 2: Azure Authentication

```
az login
az account set --subscription "Subscription-1"
```

Step 3: Terraform Initialization

```
terraform init
# Downloads Azure provider plugins
# Initializes backend (if configured)
```

Step 4: Infrastructure Planning

```
terraform plan
# Reviews ~25 resources to be created
# Validates configuration
# Shows estimated changes
```

Step 5: Infrastructure Deployment

```
terraform apply
# Creates Resource Group
# Deploys VNet and subnets
# Creates AKS cluster (~10-15 minutes)
# Sets up ACR
# Configures DNS zone
# Creates storage accounts
# Establishes network security
```

Terraform Output:

- AKS credentials command
- ACR login server
- DNS nameservers
- Load Balancer public IP
- Service URLs

5.2 Kubernetes Access Configuration

```
# Get AKS credentials
az aks get-credentials \
  --resource-group medplum-stack-dev-rg \
  --name medplum-stack-dev-aks

# Verify connection
kubectl get nodes
kubectl get namespaces
```

5.4 DNS Configuration

1. Retrieved Azure DNS nameservers from Terraform output
2. Logged into Hostinger account
3. Updated medplum.fun nameservers to Azure DNS values
4. Waited for DNS propagation (typically 1-24 hours)

5. Verified with: `dig app.medplum.fun`

5.5 Verification

Infrastructure Verification:

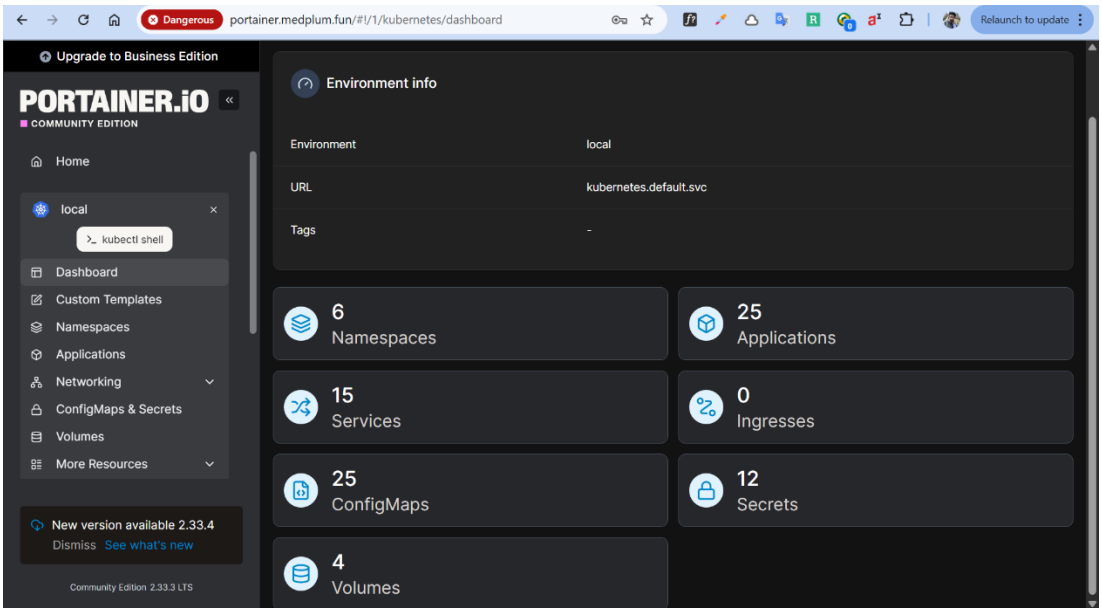
```
# Check all resources
az resource list --resource-group medplum-stack-dev-rg --output table

# Verify AKS
kubectl get nodes
kubectl get pods --all-namespaces

# Verify services
kubectl get svc --all-namespaces

# Check persistent volumes
kubectl get pvc --all-namespaces
```

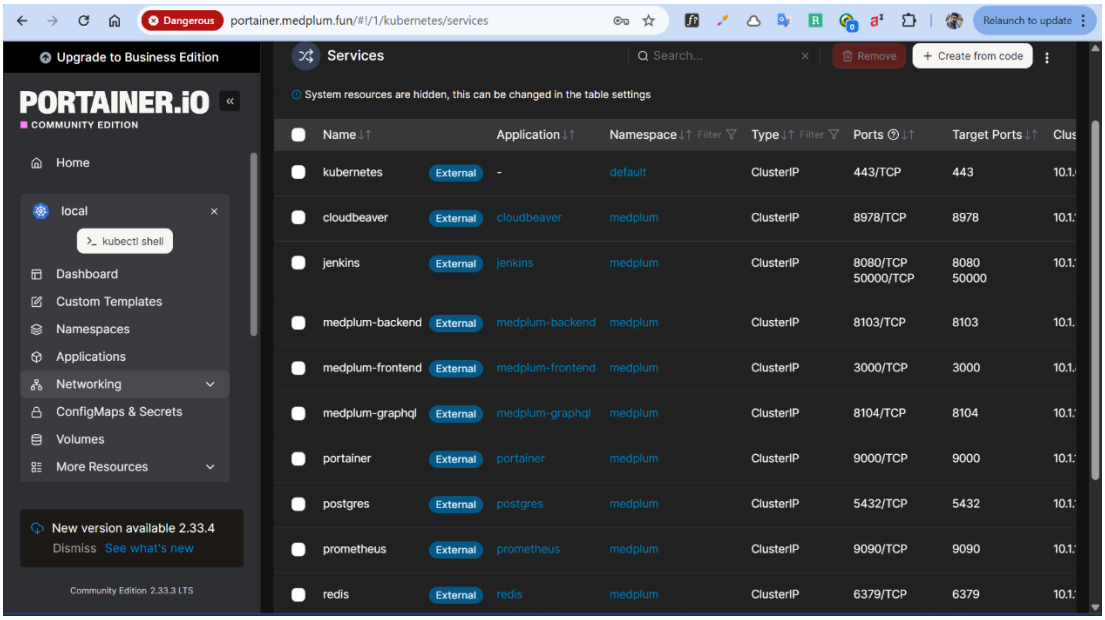
Service Accessibility:

- app.medplum.fun - Frontend accessible
- api.medplum.fun - Backend API responding
- graph.medplum.fun - GraphQL endpoint active
- portainer.medplum.fun - Management UI accessible
- prometheus.medplum.fun - Metrics available
- PostgreSQL internal connectivity verified

5.6 Portainer

Portainer is an open source pproject that can be deployed on Kubernetes to visualize and manage the Kubernetes services, deployments, configmaps through the interactive ui. Here are some screenshots of the deployments for better understanding


Dangerous
portainer.medplum.fun/#!/1/kubernetes/dashboard
Relaunch to update
Upgrade to Business Edition
PORTAINER.io
COMMUNITY EDITION
Home
local
kubectl shell
Dashboard
Custom Templates
Namespaces
Applications
Networking
ConfigMaps & Secrets
Volumes
More Resources
New version available 2.33.4
Dismiss See what's new
Community Edition 2.33.3 LTS
Environment info
Environment local
URL kubernetes.default.svc
Tags -
6 Namespaces
25 Applications
15 Services
0 Ingresses
25 ConfigMaps
12 Secrets
4 Volumes


Dangerous
portainer.medplum.fun/#!/1/kubernetes/services
Relaunch to update
Upgrade to Business Edition
PORTAINER.io
COMMUNITY EDITION
Home
local
kubectl shell
Dashboard
Custom Templates
Namespaces
Applications
Networking
ConfigMaps & Secrets
Volumes
More Resources
New version available 2.33.4
Dismiss See what's new
Community Edition 2.33.3 LTS
Services
Search...
Remove
Create from code
System resources are hidden, this can be changed in the table settings
Name
Application
Namespace
Type
Ports
Target Ports
kubernetes External - default ClusterIP 443/TCP 443
cloudbeaver External cloudbeaver medplum ClusterIP 8978/TCP 8978
jenkins External jenkins medplum ClusterIP 8080/TCP 50000/TCP 8080 50000
medplum-backend External medplum-backend medplum ClusterIP 8103/TCP 8103
medplum-frontend External medplum-frontend medplum ClusterIP 3000/TCP 3000
medplum-graphql External medplum-graphql medplum ClusterIP 8104/TCP 8104
portainer External portainer medplum ClusterIP 9000/TCP 9000
postgres External postgres medplum ClusterIP 5432/TCP 5432
prometheus External prometheus medplum ClusterIP 9090/TCP 9090
redis External redis medplum ClusterIP 6379/TCP 6379

Dangerous portainer.medplum.fun/#!/1/kubernetes/volumes

Relaunch to update

Upgrade to Business Edition

PORTAINER.io
COMMUNITY EDITION

- Home
- local
  - kubectl shell
  - Dashboard
  - Custom Templates
  - Namespaces
  - Applications
  - Networking
  - ConfigMaps & Secrets
  - Volumes
  - More Resources

New version available 2.33.4
Dismiss See what's new

Community Edition 2.33.3 LTS

Volumes

Volume list

admin

Volumes | Storage

Volumes

Search... | Remove | + Create from code

System resources are hidden, this can be changed in the table settings

Name		Namespace	Used by	Storage	Size	Created
jenkins-pvc	External	medplum	jenkins	managed-csi	5.4 GB	2025-11-10 21:56:25 by jenkins
postgres-pvc	External	medplum	postgres	default	10.7 GB	2025-11-12 15:48:08 by postgres
redis-pvc	External	medplum	redis	default	5.4 GB	2025-11-12 15:48:08 by redis
vault-pvc	External	medplum	vault	default	1.1 GB	2025-11-10 21:56:25 by vault

Items per page 10

Dangerous portainer.medplum.fun/#!/1/kubernetes/configurations?tab=secrets

Relaunch to update

Upgrade to Business Edition

PORTAINER.io
COMMUNITY EDITION

- Services
- Ingresses
- ConfigMaps & Secrets
- Volumes
- More Resources
  - Cron Jobs & Jobs
  - Service Accounts
  - Cluster Roles
  - Roles
- Cluster

Administration

- User-related

New version available 2.33.4
Dismiss See what's new

Community Edition 2.33.3 LTS

ConfigMaps & Secrets

ConfigMap & Secret lists

admin

ConfigMaps | Secrets

Secrets

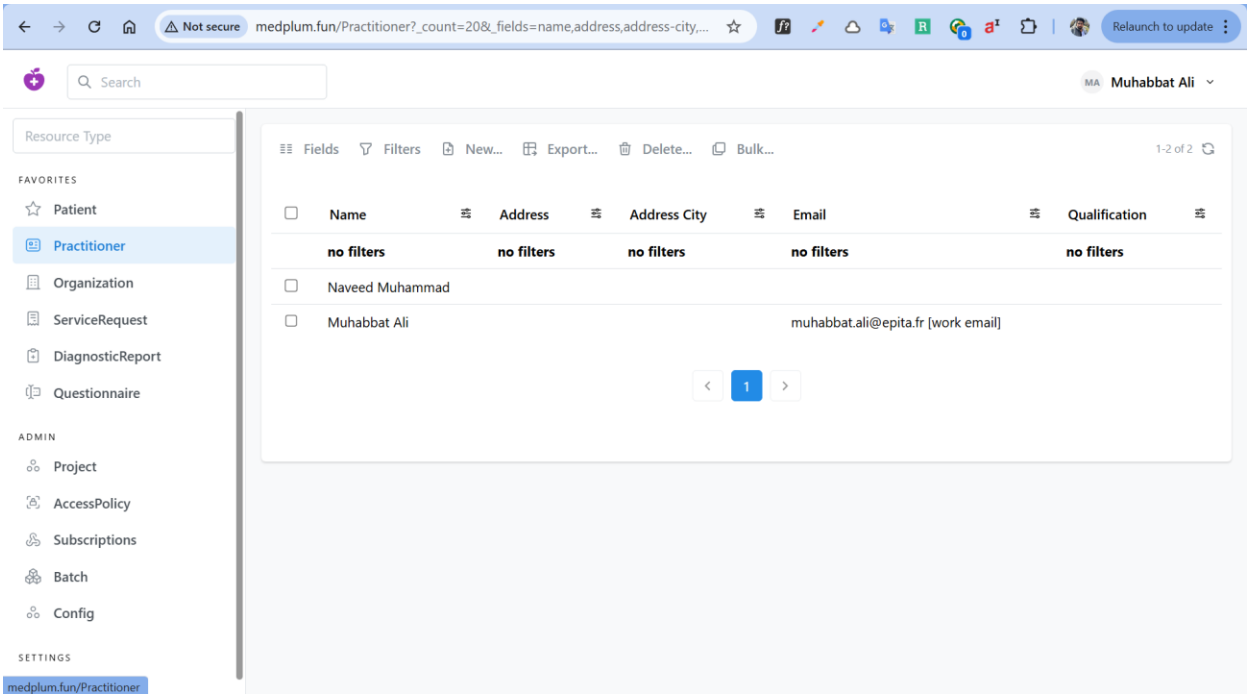
Search... | Remove | + Add with form | + Create from code

System resources are hidden, this can be changed in the table settings

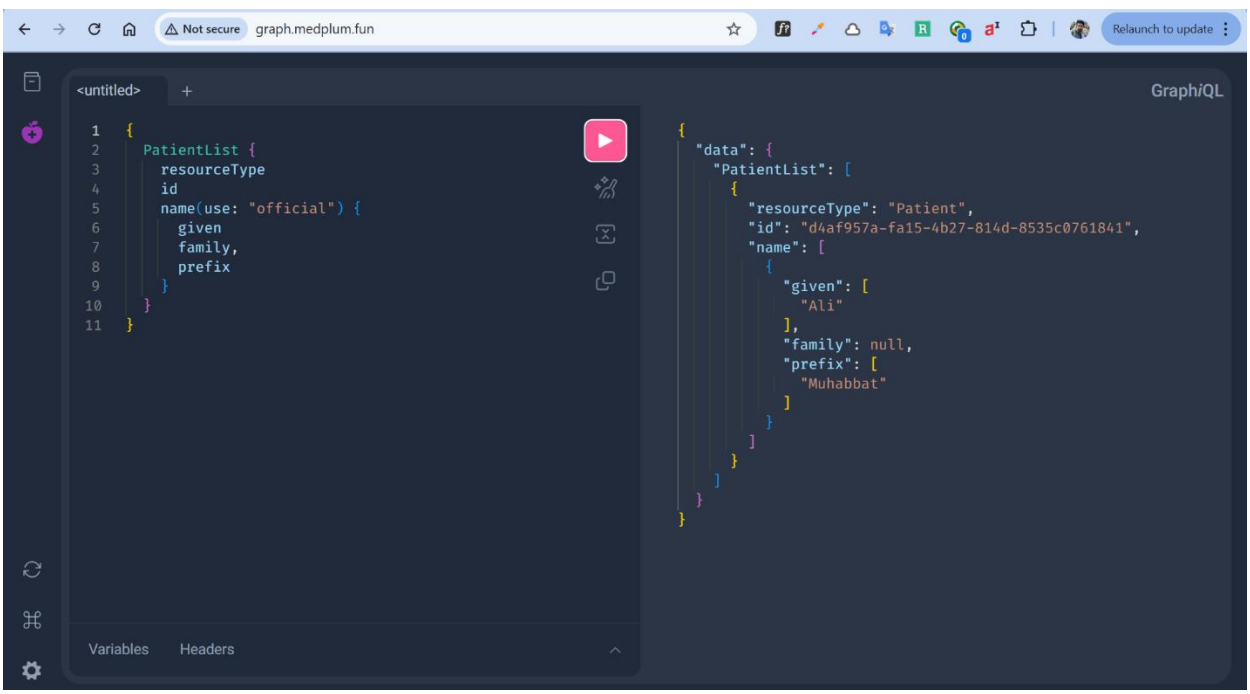
Name			Namespace	Created
vault-basic-auth	External	Unused	default	2025-11-10 16:45:11
azure-storage-account-f29fe2311bb6848ad96ca78-secret	External	Unused	medplum	2025-11-11 17:33:49
medplum-secrets	External		medplum	2025-11-12 13:59:47
vault-basic-auth	External		medplum	2025-11-10 21:56:25
sh.helm.release.v1.traefik.v1	External	Unused	traefik	2025-11-10 23:27:00

Items per page 10

5.7 Application Screenshots



Frontend Dashboard



GraphQL server

6. Security Implementation

6.1 Network Security

Network Isolation:

- All resources deployed within private VNet
- Network Security Groups control traffic flow
- Subnets segregate different service tiers
- Service endpoints for secure Azure service access

Access Control:

- Public access only through Load Balancer on approved ports
- PostgreSQL accessible only within Kubernetes cluster
- Storage account accessible only from VNet subnets
- ACR accessible only by authorized identities

6.2 Identity and Access Management

Managed Identities:

- AKS uses system-assigned managed identity
- No hardcoded credentials in code or configuration
- Automatic credential rotation by Azure
- Role assignments for minimal required permissions

RBAC (Role-Based Access Control):

- AKS role assignment: "AcrPull" for ACR access
- Network Contributor role for VNet operations
- Resource-specific permissions only

6.3 Data Security

Encryption:

- All data encrypted at rest (Azure Managed Disks)
- TLS 1.2 minimum for all communications
- HTTPS enforced for all external endpoints
- Storage account encryption enabled

Secrets Management:

- Database credentials stored in Kubernetes Secrets
- ACR credentials managed by Azure

6.4 Compliance

Azure Security Features:

- Azure Security Center recommendations followed
- Container image scanning available in ACR
- Network flow logs available (optional)
- Audit logs enabled for all Azure operations

---

8. Challenges and Solutions

8.1 Challenge: AKS Node Sizing

Issue: Initial deployment with t3.micro equivalent was under-resourced
Solution: Upgraded to Standard_B2s with 4GB RAM for stable operation
Learning: Always test with realistic workloads before finalizing VM size

8.2 Challenge: DNS Propagation Delay

Issue: DNS records not immediately accessible after deployment
Solution: Documented 24-48 hour wait time; used direct IP access for testing
Learning: Plan DNS changes in advance; use Azure Traffic Manager for zero-downtime migrations

8.3 Challenge: PostgreSQL Data Persistence

Issue: Initially, PostgreSQL data was lost on pod restarts
Solution: Implemented PersistentVolumeClaim with Azure Disk storage class
Learning: Always use persistent volumes for stateful services

8.4 Challenge: ACR Authentication from AKS

Issue: Image pull errors due to authentication failures
Solution: Configured managed identity role assignment for AcrPull
Learning: Managed identities are preferred over service principals

8.5 Challenge: Network Security Configuration

Issue: Services not accessible due to restrictive NSG rules
Solution: Carefully planned and documented required ports in NSG
Learning: Test network connectivity systematically per service

---

9. Future Improvements

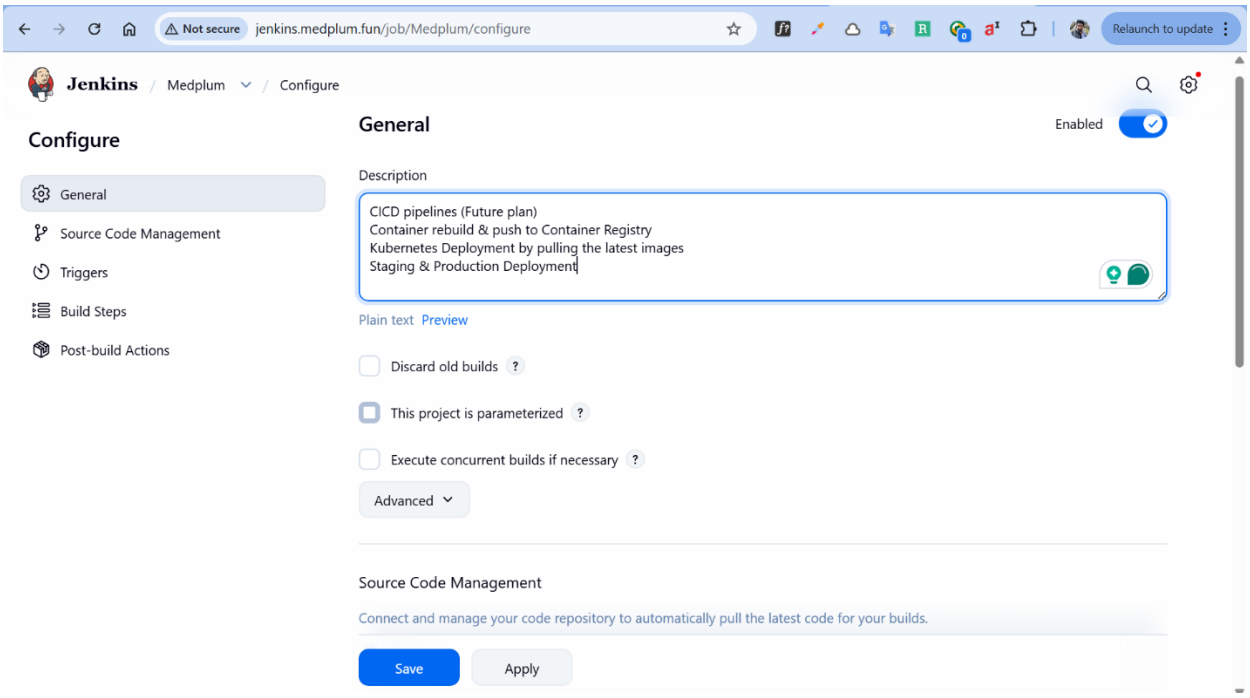
9.1 CI/CD Pipeline Infrastructure

Current State: Basic Azure DevOps pipeline for image building

Planned Enhancements:

9.1.1 Complete CI/CD Pipeline Architecture

```
Developer Commits → GitHub/Azure Repos
         ↓
    Azure Pipeline Triggered
         ↓
      ┌──────────────────────────┐
      │  Build Stage             │
      │ - Code checkout          │
      │ - Unit tests             │
      │ - Security scanning      │
      │ - Docker build           │
      └──────────────────────────┘
         ↓
      ┌──────────────────────────┐
      │  Test Stage              │
      │ - Integration tests      │
      │ - Container scanning     │
      │ - Push to ACR            │
      └──────────────────────────┘
         ↓
      ┌──────────────────────────┐
      │  Deploy Stage            │
      │ - Update K8s manifests   │
      │ - Rolling deployment     │
      │ - Health checks          │
      │ - Smoke tests            │
      └──────────────────────────┘
         ↓
      ┌──────────────────────────┐
      │  Post-Deploy             │
      │ - Notify team            │
      │ - Update documentation   │
      │ - Performance tests      │
      └──────────────────────────┘
```



Jenkins server

Implementation Plan:

1. **Automated Testing**
   - Unit tests run on every commit
   - Integration tests before deployment
   - End-to-end tests post-deployment
   - Code coverage requirements (80% minimum)
2. **Security Scanning**
   - Container image vulnerability scanning (Trivy/Aqua)
   - Dependency vulnerability checks
   - Secret detection in code
   - Infrastructure security validation
3. **Deployment Strategies**
   - Blue-Green deployments for zero downtime
   - Canary releases for gradual rollout
   - Automatic rollback on failure
   - Feature flags for controlled releases
4. **Environment Management**
   - Separate pipelines for dev/staging/production
   - Environment-specific configurations

  - Approval gates for production deployments
  - Automated environment provisioning
5. **Monitoring Integration**
  - Deployment tracking in monitoring tools
  - Automated performance regression detection
  - Log aggregation from deployment events
  - Alert rules for deployment failures

Expected Benefits:

- Faster deployments
- Reduced human error
- Consistent deployments

9.2 Azure Storage Backup Plans

Current State: Manual backup capability exists but not automated

Planned Implementation:

9.2.1 Blob Storage Backup Strategy

```
┌──────────────────────────────────────────────────┐
│          Azure Blob Storage Backup                │
├──────────────────────────────────────────────────┤
│                                                  │
│   ┌──────────────────────────────────────┐       │
│   │ Primary Storage Account              │       │
│   │ (medplumstackdevsa)                  │       │
│   │                                      │       │
│   │ Containers:                          │       │
│   │ ├── medplum-data (hot tier)          │       │
│   │ └── backups (cool tier)              │       │
│   └──────────────────────────────────────┘       │
│                  │                               │
│                  │ Geo-replication               │
│                  ↓                               │
│   ┌──────────────────────────────────────┐       │
│   │ Backup Storage Account               │       │
│   │ (Different region)                   │       │
│   │                                      │       │
│   │ ├── Daily snapshots (7 days)         │       │
│   │ ├── Weekly snapshots (4 weeks)       │       │
│   │ └── Monthly snapshots (12 months)    │       │
│   └──────────────────────────────────────┘       │
└──────────────────────────────────────────────────┘
```

Implementation Components:

1. **Azure Blob Versioning**
   - Enable versioning on primary containers
   - Automatic version creation on overwrites
   - Version lifecycle management
   - Point-in-time recovery capability
2. **Soft Delete**
   - 30-day retention for deleted blobs
   - Protection against accidental deletion
   - Easy restoration process
3. **Lifecycle Management Policies**

```yaml
Rules:
- Name: "Move-to-Cool-After-30-Days"
  Enabled: true
  Type: Lifecycle
  Definition:
    Actions:
      BaseBlob:
        TierToCool:
          DaysAfterModificationGreaterThan: 30
        TierToArchive:
          DaysAfterModificationGreaterThan: 90
        Delete:
          DaysAfterModificationGreaterThan: 365
```

4. **Cross-Region Replication**
   - Geo-redundant storage (GRS) for critical data
   - Asynchronous replication to secondary region
   - Read access from secondary region (RA-GRS)
   - Automatic failover capability

9.2.2 Azure Storage Functions for Automated Backups

Architecture:

```
Scheduled Trigger (Daily at 2 AM UTC)
         ↓
┌──────────────────────────┐
│  Azure Function App      │
│  (Consumption Plan)      │
│                          │
│  Function: BackupBlobs   │
│  Runtime: Python 3.11    │
│  Trigger: Timer          │
└──────────────────────────┘
         ↓
    Executes Backup Logic
         ↓
┌──────────────────────────────────┐
│  Backup Operations               │
│  1. List all blobs in source     │
│  2. Copy to backup account       │
│  3. Tag with timestamp           │
│  4. Validate copy success        │
│  5. Log to App Insights          │
│  6. Send notification            │
└──────────────────────────────────┘
         ↓
    Backup Complete
         ↓
┌──────────────────────────┐
│  Notification            │
│  - Email to team         │
│  - Slack webhook         │
│  - Azure Monitor alert   │
└──────────────────────────┘
```

Function Implementation Plan:

1. **Blob Backup Function** (Python)

```python
# Pseudo-code
def blob_backup_function(timer_trigger):
    - Connect to source storage account
    - Get list of containers to backup
```

```
    - For each container:
        * Copy blobs to backup account
        * Add metadata (backup date, source)
        * Verify integrity
    - Generate backup report
    - Send notifications
```

2. **Database Backup Function**

```
def postgres_backup_function(timer_trigger):
    - Connect to PostgreSQL in AKS
    - Execute pg_dump command
    - Compress backup file
    - Upload to Azure Blob Storage
    - Rotate old backups (keep last 30)
    - Verify backup completeness
    - Log backup metrics
```

3. **Cleanup Function**

```
def cleanup_old_backups(timer_trigger):
    - List all backup blobs
    - Filter by retention policy
    - Delete expired backups
    - Log deletions
    - Update metrics
```

Backup Schedule:

Backup Type	Frequency	Retention	Storage Tier
Database Full	Daily 2 AM	30 days	Cool
Database Incremental	Every 6 hours	7 days	Hot
Blob Storage	Daily 3 AM	90 days	Cool → Archive
Configuration	Weekly	90 days	Cool
Disk Snapshots	Daily	7 days	Standard

Monitoring and Alerting:

- Function execution metrics in Application Insights
- Backup success/failure alerts
- Storage capacity monitoring
- Cost tracking per backup type
- Restore time metrics (SLA tracking)

9.2.3 Azure Disk Backup Strategy

Disk Backup Architecture:

```
┌─────────────────────────────────────────────────────┐
│        Azure Disk Backup Service                    │
├─────────────────────────────────────────────────────┤
│                                                     │
│  Managed Disks (AKS & PostgreSQL)                   │
│        ↓                                            │
│  ┌──────────────────────────────────────────────┐   │
│  │   Backup Vault                               │   │
│  │   (Geo-redundant storage)                    │   │
│  │                                              │   │
│  │   Backup Policies:                           │   │
│  │   ├── Hourly snapshots (24 hours)            │   │
│  │   ├── Daily snapshots (7 days)               │   │
│  │   ├── Weekly snapshots (4 weeks)             │   │
│  │   └── Monthly snapshots (12 months)          │   │
│  └──────────────────────────────────────────────┘   │
│        ↓                                            │
│  ┌──────────────────────────────────────────────┐   │
│  │   Recovery Services Vault                    │   │
│  │   - Point-in-time recovery                   │   │
│  │   - Cross-region restore                     │   │
│  │   - Application-consistent backups           │   │
│  └──────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────┘
```

Implementation Steps:

1. **Azure Backup Vault Setup**

```python
# Create Backup Vault
az backup vault create \
  --resource-group medplum-stack-dev-rg \
  --name medplum-backup-vault \
  --location francecentral
```

2. **Disk Backup Policy Configuration**

   - Policy 1: PostgreSQL Disk (frequent backups)
     - Every 4 hours
     - Retain for 7 days
     - Instant restore enabled
   - Policy 2: Application Disks (standard backups)
     - Daily at 1 AM
     - Retain for 30 days

    - Weekly for 12 weeks
    - Monthly for 12 months

3. **Automated Snapshot Management**
    - Azure Managed Disk snapshots
    - Incremental snapshots (cost-effective)
    - Automatic rotation based on policy
    - Cross-region copy for DR
4. **Backup Testing**
    - Monthly restore test to verify backup integrity
    - Automated restore time measurement
    - Test environment provisioning from backups
    - Documentation of restore procedures

Disaster Recovery Capabilities:

- **RPO (Recovery Point Objective)**: 4 hours for critical data
- **RTO (Recovery Time Objective)**: 2 hours for full recovery
- **Cross-region restore**: Secondary region (West Europe)
- **Backup encryption**: Azure encryption at rest

10. Conclusion

10.1 Project Summary

This project successfully demonstrated the deployment of a complete healthcare platform on Microsoft Azure cloud infrastructure. Using Infrastructure as Code principles with Terraform and configuration management with Ansible, we created a reproducible, scalable, and maintainable cloud environment.

Key Accomplishments:

Infrastructure Automation: All Azure resources provisioned via Terraform, enabling consistent and repeatable deployments

Containerization: Eight services deployed on Azure Kubernetes Service with proper isolation and resource management

Networking: Secure VNet configuration with proper subnet segmentation and network security groups

Storage: Persistent storage implemented for stateful services using Azure Managed Disks

DNS Management: Custom domain (medplum.fun) configured with Azure DNS for all service endpoints

Monitoring: Comprehensive monitoring setup with Azure Monitor, Prometheus, and Grafana

Security: Network isolation, managed identities, encrypted storage, and proper access controls

Documentation: Complete documentation enabling future team members to understand and maintain the infrastructure

10.2 Learning Outcomes

Technical Skills Developed:

- Azure cloud platform proficiency
- Infrastructure as Code with Terraform
- Kubernetes orchestration and management
- Container registry and image management
- Network architecture and security
- DNS configuration and management
- Monitoring and observability
- Ansible automation

Best Practices Learned:

- Always use Infrastructure as Code for reproducibility
- Implement proper network segmentation
- Use managed identities over hardcoded credentials
- Plan for data persistence in containerized environments
- Monitor everything from day one
- Document architecture and deployment procedures
- Consider costs during architecture design
- Test disaster recovery procedures

10.3 Business Value

For Healthcare Operations:

- **Scalability**: Platform can grow with user demand
- **Reliability**: High availability design minimizes downtime
- **Security**: Proper isolation and encryption protect patient data
- **Compliance Ready**: Foundation for HIPAA and GDPR compliance
- **Cost Effective**: Optimized resources keep operational costs low
- **Maintainable**: IaC approach enables easy updates and changes

10.5 Final Thoughts

This project demonstrates that modern cloud infrastructure can be both powerful and accessible. By leveraging Azure's managed services and automation tools, we've created an enterprise-grade platform that would have required significant manual effort just a few years ago.

Key Takeaway: Infrastructure as Code is not just about automation, it's about creating knowledge that can be shared, version-controlled, and continuously improved. This project embodies that principle, providing both a working system and a blueprint for future improvements.

Appendices

Appendix A: Resource Naming Conventions

All resources follow the pattern: `{project}-{environment}-{resource-type}`

Examples:

- `medplum-stack-dev-rg` (Resource Group)
- `medplum-stack-dev-aks` (AKS Cluster)
- `medplum-stack-dev-vnet` (Virtual Network)

Appendix B: Key Commands Reference

Terraform:

```
terraform init                    # Initialize
terraform plan                    # Preview changes
terraform apply                   # Deploy infrastructure
terraform destroy                 # Remove all resources
terraform output                  # Show outputs
terraform state list              # List managed resources
```

Azure CLI:

```
az login                          # Authenticate
az account list                   # List subscriptions
az aks get-credentials            # Get kubeconfig
az resource list                  # List resources
az acr login                      # Login to registry
```

Kubernetes:

```
kubectl get pods -A               # List all pods
kubectl get svc -n medplum        # List services
kubectl logs -f <pod>             # View logs
```

```
kubectl describe pod <pod>        # Pod details
kubectl exec -it <pod> -- bash    # Shell access
```

Appendix C: Troubleshooting Quick Reference

Issue	Command	Resolution
Pod not starting	`kubectl describe pod <name>`	Check events section
Service not accessible	`kubectl get svc`	Verify LoadBalancer IP
DNS not resolving	`dig app.medplum.fun`	Check DNS propagation
High costs	`az consumption usage list`	Review resource usage
AKS not accessible	`az aks get-credentials`	Re-download kubeconfig

Glossary

AKS: Azure Kubernetes Service - Managed Kubernetes offering from Microsoft Azure

ACR: Azure Container Registry - Docker image registry service

NSG: Network Security Group - Firewall rules for Azure resources

VNet: Virtual Network - Isolated network in Azure

PVC: Persistent Volume Claim - Kubernetes storage request

IaC: Infrastructure as Code - Managing infrastructure through code files

RBAC: Role-Based Access Control - Permission system based on roles

SKU: Stock Keeping Unit - Service tier/size in Azure

LoadBalancer: Service that distributes traffic across multiple servers

Managed Identity: Azure service for automatic credential management

Helm: Package manager for Kubernetes applications

Terraform: Infrastructure as Code tool for cloud provisioning

---